ViToxic-Analyzer

Tài liệu học qua project: hiểu bài toán, tự giải thích được mã nguồn, kiểm chứng kết quả và thực hiện thay đổi có kiểm soát.

Biên soạn ngày 16/10/2026 theo source và báo cáo hiện có trong workspace. Các điểm số bên dưới là kết quả đã lưu, không phải một lần huấn luyện hoặc đánh giá mới khi viết tài liệu.

1. Cách học với tài liệu này

Mục tiêu không chỉ là chạy notebook đến ô cuối. Sau mỗi phần, bạn cần trả lời được: **đầu vào là gì, xử lý thế nào, vì sao chọn cách này, đầu ra ở đâu, và làm sao biết nó đúng?**

Học theo vòng lặp: đọc khái niệm → mở hàm được chỉ dẫn → dự đoán kết quả trước khi chạy → thực hành nhỏ → đối chiếu bằng chứng → tự giải thích lại. Mỗi buổi chỉ cần hoàn thành một mắt xích, không cần hiểu toàn bộ Transformer ngay buổi đầu.

Tài liệu dùng Markdown để đặt cạnh source và cập nhật cùng project. Các đường dẫn trong tài liệu tính từ thư mục gốc dự án. Lệnh terminal chạy tại thư mục gốc với đúng môi trường Python đã kích hoạt. Bài thực hành dùng dữ liệu tổng hợp hoặc đọc artifact; thí nghiệm mới phải có thư mục riêng.

Lộ trình đề nghị

Buổi	Đọc	Việc làm	Sản phẩm chứng minh đã hiểu
1	Phần 2–4	Đi từ README đến từng module	Tự vẽ đường đi của một bình luận
2	Phần 5–6	Khám phá schema, audit, preprocessing	Bảng đầu vào/đầu ra của 8 câu tổng hợp
3	Phần 7	Phân biệt âm tiết, từ, subword, tensor	Giải thích `input_ids` và `attention_mask`
4	Phần 8	Hiểu baseline	Tính TF-IDF trên ví dụ nhỏ, đọc ba giá trị C
5	Phần 9	Đọc model và tính softmax	Ghi kích thước tensor qua từng lớp
6	Phần 10	Theo dõi một bước training	Giải thích loss, gradient và optimizer step
7	Phần 11	Đọc config, history, checkpoints	Phân biệt tái lập, reload và resume
8	Phần 12	Tính metrics và bootstrap	Tính Macro-F1 từ confusion matrix
9	Phần 13	Đọc các lỗi và learning curves	Viết nhận xét có bằng chứng và giới hạn
10	Phần 14–15	Gửi request API, đọc tests	Giải thích HTTP flow và các tầng kiểm chứng
11	Phần 16–18	Chọn một bài mở rộng phù hợp	Một ghi chú thí nghiệm, không thay kết quả đã khóa

Bạn có thể chia mỗi buổi thành nhiều buổi nhỏ. Checkbox tự đánh giá ở cuối do chính bạn đánh dấu.

2. Project giải quyết bài toán gì?

Project nhận một bình luận tiếng Việt và phân loại vào một trong ba nhãn:

ID trong code	Nhãn	Cách hiểu nhập môn
0	CLEAN	Nội dung được gán nhãn không thuộc hai nhóm độc hại còn lại
1	OFFENSIVE	Nội dung xúc phạm theo hệ thống nhãn của bộ dữ liệu
2	HATE	Nội dung thù ghét theo hệ thống nhãn của bộ dữ liệu

Đây là **học có giám sát, phân loại đa lớp, một nhãn cho mỗi mẫu**. Nhãn chuẩn là `label_id`; model học từ các cặp văn bản–nhãn. Mô tả ở bảng chỉ giúp định hướng: khi tự gán nhãn, phải dùng hướng dẫn annotation của bộ dữ liệu; không tự đồng nhất mọi từ tục với HATE. Code không chứa một bộ quy tắc ngôn ngữ đầy đủ để phân biệt hai lớp này.

Model học quan hệ thống kê từ dữ liệu, không tự xác minh ý định người viết. Một câu trích dẫn, nói đùa hoặc phản bác lời xúc phạm có thể chứa từ tương tự câu độc hại. Đây là lý do cần phân tích ngữ cảnh thay vì chỉ đếm từ khóa.

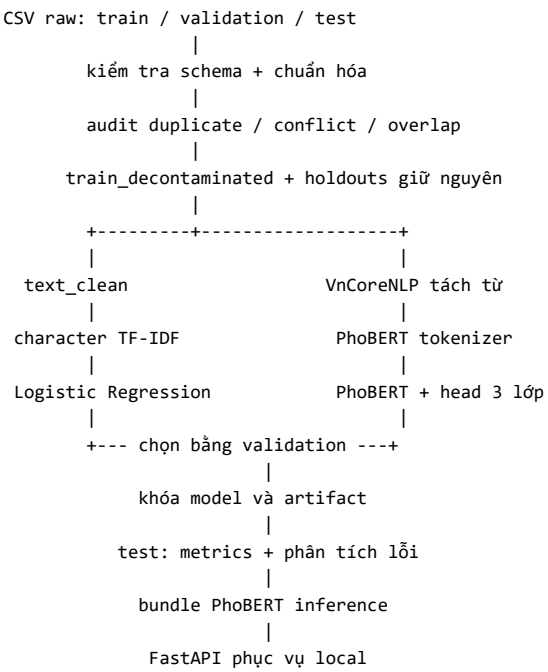
Project hiện có hai hướng mô hình:

• **Baseline:** character TF-IDF + Logistic Regression, giúp có mốc so sánh rõ ràng.

• **PhoBERT:** mô hình ngôn ngữ tiền huấn luyện được fine-tune cho ba nhãn, là model phục vụ API hiện tại.

Đầu ra API là nhãn, điểm softmax của ba lớp và thông tin cắt ngắn đầu vào. Project hiện chưa có giao diện quản trị moderation, cơ chế tự xóa bình luận hoặc triển khai public đã xác minh.

3. Bức tranh toàn bộ hệ thống



Đường đi khi phục vụ một câu mới ngắn hơn: HTTP JSON → kiểm tra đầu vào → preprocessing → VnCoreNLP → tokenizer → PhoBERT → logits → softmax/argmax → JSON. Không fit TF-IDF và không cập nhật trọng số trong request.

Phân biệt ba thao tác: **training** cập nhật tham số; **evaluation** so dự đoán với nhãn có sẵn; **inference** tạo dự đoán cho đầu vào. Evaluation sử dụng inference rồi tính thêm metrics.

4. Kiến thức nền và bản đồ mã nguồn

4.1. Kiến thức cần nắm dần

Nhóm	Cần hiểu trong project	Chỗ thực hành
Python	Hàm, class, import, exception, dataclass, context manager	`PreprocessingConfig`, `TrainConfig`, `Predictor`
File và dữ liệu	CSV, JSON, đường dẫn, UTF-8, hash	`load_split`, `load_processed`, `sha256`
NumPy/Pandas	Array, DataFrame, boolean mask, groupby, quantile	Audit, class weights, bootstrap
Đại số tuyến tính	Vector, ma trận, phép nhân, shape	TF-IDF, linear layer, attention
Xác suất	Phân phối, kỳ vọng, log, lấy mẫu có hoàn lại	Softmax, cross-entropy, bootstrap
Tối ưu	Đạo hàm, chain rule, gradient descent	`backward`, AdamW, clipping
ML	Generalization, overfitting, train/validation/test	Chọn checkpoint và đánh giá
Kỹ thuật phần mềm	Test, cấu hình, môi trường, API, lifecycle	Stage 2–4

Không cần học hết lý thuyết trước khi mở code. Ví dụ, học boolean mask khi đọc lọc train; học chain rule khi tới `loss.backward()`.

4.2. Mở file nào để hiểu việc nào?

File	Trách nhiệm và điểm đọc chính
`src/data_loader.py`	`LABELS`, `load_split`, `class_weights`, sampler; có wrapper DataLoader, nhưng training chính dùng `dataset.make_loader`
`src/preprocessing.py`	`PreprocessingConfig`, `preprocess_text`: biến raw text thành text sạch
`src/stage1.py`	`fingerprint`, `audit`, `run`: audit, EDA và lọc train
`src/segmentation.py`	`prepare_segmenter`, `Segmenter`, `segment_frame`: Java assets và cache tách từ
`src/dataset.py`	`ViHSDDataset`, `make_loader`: tokenizer, batching, padding
`src/baseline.py`	`run`: thử C, fit TF-IDF trên train, lưu baseline tốt nhất
`src/model.py`	`PhoBERTClassifier`, `load_checkpoint_model`: forward và reload offline
`src/config.py`	`TrainConfig.validate`: tham số và ràng buộc
`src/train.py`	`train_epoch`, `evaluate`, `EarlyStopping`, `run`: vòng lặp học
`src/experiment.py`	Schema processed, metrics, SHA-256, ghi JSON qua file tạm
`src/tune.py`	Tạo kế hoạch search, chạy trial, tổng hợp lựa chọn
`src/evaluate.py`	Khóa evaluation, inference test, bootstrap và báo cáo
`src/serving.py`	Kiểm tra bundle, `Predictor.predict`, pipeline inference

File	Trách nhiệm và điểm đọc chính
`src/api.py`	Schema HTTP, body limit, lifespan, điều phối worker

`notebooks/` là điểm thao tác và trình bày; `src/` chứa logic tái sử dụng; `scripts/` xây notebook/báo cáo, đóng gói và kiểm chứng; `outputs/` lưu bằng chứng. Đừng xem notebook là nơi duy nhất định nghĩa thuật toán.

`requirements/*.txt` quy định khoảng phiên bản cài đặt; `requirements/locks/` ghi môi trường cụ thể. Một khoảng phiên bản không đảm bảo hai lần cài giống nhau. `.venv` cách ly thư viện Python, còn Java vẫn cần có để chạy VnCoreNLP. Chi tiết cấu trúc ở `docs/PROJECT_STRUCTURE.md`.

5. Stage 1 — Dữ liệu, leakage và EDA

5.1. Schema và chia tập

CSV raw yêu cầu `free_text`, `label_id`. `load_split` tạo `row_id` dạng `train:0`, `validation:0`, `test:0` để giữ liên kết giữa một hàng dữ liệu và dự đoán của nó. Processed có thêm `text_clean`; loader processed kiểm tra prefix split và row ID không trùng.

Tập	Số mẫu đã ghi nhận	Vai trò
Train gốc	24.048	Dữ liệu ban đầu để audit
Train decontaminated	21.292	Dữ liệu thực dùng huấn luyện
Validation	2.672	Chọn C, hyperparameters và checkpoint
Test	6.680	Đánh giá cuối sau khi khóa model

Train dạy model; validation giúp quyết định cách dạy; test đo kết quả sau quyết định. Nếu xem lỗi test rồi đổi learning rate và tiếp tục báo điểm test như đánh giá độc lập, bạn đã dùng test như validation.

`keep_default_na=False` giữ các chuỗi như `NA`, `null` đúng là văn bản. Chuỗi rỗng được kiểm tra riêng. Nếu tự chuyển missing value thành chuỗi `nan`, model có thể học một tín hiệu giả do pipeline tạo ra.

5.2. Duplicate, conflict, overlap khác nhau thế nào?

• **Duplicate:** nhiều hàng có cùng khóa văn bản chuẩn hóa.

• **Conflict:** cùng khóa nhưng có nhiều nhãn khác nhau.

• **Overlap:** khóa xuất hiện ở hơn một split.

• **Leakage:** thông tin ngoài giao thức học lọt vào quyết định/model, làm đánh giá lạc quan. Overlap là một nguồn rủi ro leakage.

Khóa so sánh là preprocessing mặc định rồi `casefold` và gom khoảng trắng. Nó không phát hiện mọi câu diễn đạt lại cùng ý nghĩa. Casefold phục vụ matching; không có nghĩa model luôn nhận văn bản lowercase.

Trong `stage1.run`, train được loại duplicate, nhóm conflict, overlap holdout và text rỗng. Nếu một hàng có nhiều lý do, lý do ghi sau ghi đè lý do trước: **empty_text > holdout_overlap > conflicting_train_labels > duplicate_train**. `train_exclusions.csv` ghi lý do chính, không phải mọi nguyên nhân đồng thời. Duplicate không conflict giữ lần xuất hiện đầu nếu không có lý do loại khác; nhóm conflict bị loại toàn bộ.

Validation/test được giữ nguyên. Pipeline có đọc holdout để audit và loại exact overlap khỏi train; không nên mô tả là “chưa từng đụng tới test”. Điểm phân biệt là không dùng điểm model trên test để chọn cấu hình. Giao thức

lọc train này phải được công bố khi so benchmark dùng train chính thức.

5.3. EDA giúp quyết định điều gì?

EDA là khám phá dữ liệu: phân bố nhãn, độ dài, n-grams, word clouds. Trong project, EDA nội dung và hình Stage 1 dùng **train gốc**, còn model học train decontaminated. Class weights có báo cáo cho cả hai; training tính từ train thực dùng.

Độ dài trong Stage 1 đếm đơn vị tách bởi khoảng trắng. Với tiếng Việt, đó chưa chắc là số từ, càng không phải số token PhoBERT. Word cloud biểu diễn tần suất, không chứng minh quan hệ nhân quả hoặc quy tắc phân loại.

Thực hành: mở `outputs/stage1/audit.json`, tìm số hàng trước/sau lọc và đối chiếu tổng lý do loại với chênh lệch. Giải thích vì sao 92 nhóm normalized text chung validation/test có thể tương ứng 95 dòng test overlap: số nhóm và số hàng là hai đại lượng khác nhau.

Đạt khi: bạn giải thích được vì sao không gộp ba split rồi chia lại, và vì sao không "sửa cho đúng" nhãn conflict bằng phỏng đoán.

6. Preprocessing — Làm sạch nhưng giữ tín hiệu

`preprocess_text` thực hiện theo thứ tự: Unicode NFC → xử lý một số ký tự ẩn → mask URL → mask mention → bỏ dấu # đầu hashtag → lowercase tùy chọn → teen code tùy chọn → gom whitespace.

Xử lý	Ví dụ/ý nghĩa	Đánh đổi
Unicode NFC	Chuẩn hóa các biểu diễn dấu tương đương	Giảm khác biệt mã hóa, không sửa chính tả
URL → `urltoken`	Giữ tín hiệu "có liên kết"	Mất danh tính URL
Mention → `usertoken`	Giữ tín hiệu "có nhắc người"	Hai câu khác người có thể cùng text sạch
`#xin_chào` → `xin_chào`	Giữ nội dung hashtag	Bỏ tín hiệu ký hiệu hashtag
Giữ emoji/dấu câu/số	Giữ sắc thái và biểu đạt	Mô hình phải tự học cách dùng
Lowercase/teen code mặc định tắt	Tránh thay đổi mơ hồ	Vẫn có biến thể cách viết

`urltoken` và `usertoken` là chuỗi thông thường, không phải special token được thêm vào vocabulary. Teen code chỉ có từ điển nhỏ `ko/khong/hok/dc`; không suy ra code xử lý mọi tiếng lóng.

Thử trong Python tại root:

Trong đoạn mã PDF, emoji được biểu diễn bằng escape Unicode \U0001f60a để hiển thị và sao chép ổn định.

```
from src.preprocessing import preprocess_text, PreprocessingConfig

text = '  @ban Chúc bạn vui! \U0001f60a https://example.com #HọcTập  '
print(preprocess_text(text))
# usertoken Chúc bạn vui! \U0001f60a urltoken HọcTập
print(preprocess_text('ko ổn', PreprocessingConfig(normalize_teencode=True)))
# không ổn
assert preprocess_text(preprocess_text(text)) == preprocess_text(text)
```

Assert cuối kiểm tra **idempotence** trên ví dụ: làm sạch lần hai không đổi kết quả. Kiểm tra thêm văn bản chỉ có whitespace, Unicode dấu tổ hợp, emoji, URL, mention, hashtag, số và teen code. Dự đoán output trước khi chạy.

Câu hỏi: tại sao xóa hết dấu câu có thể làm mất tín hiệu? Tại sao đổi preprocessing khi inference mà không đổi training gây sai lệch đầu vào? Nếu thay preprocessing cho nghiên cứu mới, phải tái tạo processed data/cache tương ứng và dùng run mới.

7. Từ văn bản đến tensor

7.1. Tách từ khác tokenization

Trong tiếng Việt, dấu cách thường phân cách âm tiết. Minh họa "sinh viên" có thể được biểu diễn thành `sinh_viên` sau tách từ; output cụ thể phải xem VnCoreNLP thực chạy. Segmenter tìm ranh giới từ. PhoBERT tokenizer tiếp tục biến văn bản đã tách từ thành các subword và ID số.

Vì vậy, chuỗi xử lý có ba mức: **text sạch → text tách từ → token IDs**. Không dùng `text.split()` thay cho VnCoreNLP rồi coi là cùng pipeline.

`Segmenter` dùng VnCoreNLP với annotator `wseg`, cần Java và ba assets được khai báo trong `ASSETS`. Wrapper có thể thay thư mục hiện hành; code khôi phục cwd để đường dẫn tương đối không bị lệch.

7.2. Cache đúng phải nhận diện đúng đầu vào

`segment_frame` tạo SHA-256 từ phiên bản cache, hash assets và danh sách `(row_id, text_clean)` theo thứ tự. Thay text, thứ tự/ID hoặc assets sẽ đổi khóa. Cache lưu kết quả công đoạn tốn thời gian; nó không làm model thông minh hơn.

SHA-256 giúp nhận diện nội dung file đã thay đổi, không chứng minh bộ dữ liệu là bản phát hành gốc hay nội dung đúng. Đây là khác biệt giữa **integrity** và **nguồn gốc/độ đúng**.

7.3. Dataset, batch, padding và truncation

`ViHSDDataset` tokenizes trước, chưa padding. `make_loader` gom các mẫu rồi dùng `DataCollatorWithPadding`, làm tròn độ dài padding lên bội số 8. Training shuffle nếu không dùng sampler; validation không shuffle theo nhánh này.

• `input_ids`: ma trận số nguyên `[B, L]`, mỗi số là ID token.

• `attention_mask`: ma trận `[B, L]`, cho encoder biết vị trí thực và vị trí padding.

• `labels`: vector `[B]`, mỗi phần tử là 0/1/2.

• **Padding:** thêm vị trí để các câu trong batch có cùng chiều dài.

• **Truncation:** cắt token vượt giới hạn, có thể mất thông tin cuối câu.

`max_length=128` tính cả special tokens, không phải 128 ký tự hoặc 128 từ. `token_length_report` đo độ dài trước cắt và tỷ lệ vượt 128/256 để thấy đánh đổi. Tăng length tốn bộ nhớ và tính toán; attention chuẩn có thành phần chi phí tăng theo bình phương chiều dài.

Thực hành: đọc `test_dynamic_padding_and_truncation`; ghi shape dự kiến của hai câu dài khác nhau trước khi đọc assertion. Khi xem câu thật, so ba con số: whitespace units, số từ sau segmentation và số token.

Đạt khi: bạn giải thích được một câu dưới 5.000 ký tự vẫn có thể bị cắt tại 128 token khi gọi API.

8. Baseline — TF-IDF + Logistic Regression

Baseline trong `src/baseline.py` nhận `text_clean` trực tiếp, không qua VnCoreNLP. `TfidfVectorizer` dùng character n-grams 3–5, `min_df=2`, tối đa 100.000 features, `sublinear_tf=True`. Các mặc định còn lại của

vectorizer vẫn có hiệu lực; riêng vectorizer mặc định lowercase, dù preprocessing chung không lowercase.

N-gram ký tự là cửa sổ liên tiếp n ký tự. Chuỗi `abcd` có hai 3-grams là `abc`, `bcd`. Cách này có thể giữ tín hiệu từ mảnh từ và cách viết biến thể, nhưng không biểu diễn ngữ cảnh như encoder Transformer.

Với mặc định IDF smoothing của vectorizer:

```
tf(t,d)  = 1 + log(count(t,d)) khi count > 0
idf(t)   = log((1 + N) / (1 + df(t))) + 1
giá trị  = tf(t,d) × idf(t), sau đó chuẩn hóa L2 mỗi vector
```

`N` là số văn bản train; `df(t)` là số văn bản train chứa feature t. Feature phổ biến khắp tập ít phân biệt hơn feature xuất hiện chọn lọc. Ma trận này thường sparse vì mỗi câu chỉ có một phần nhỏ vocabulary.

Logistic Regression học trọng số cho features để dự đoán lớp. Dù tên có "Regression", đây là classifier. `C` điều khiển nghịch đảo độ mạnh regularization: C nhỏ hơn tương ứng regularization mạnh hơn. Project thử `0.5, 1.0, 2.0`, `class_weight='balanced'`, chọn bằng validation Macro-F1; C=2 được chọn trong artifact Colab.

Fit vectorizer chỉ trên train để vocabulary/IDF không học thống kê holdout. Validation chỉ đi qua transform và predict. Pipeline gói vectorizer + classifier giúp giữ đúng thứ tự khi save/load.

Thực hành: dùng ba câu tổng hợp, tự liệt kê một vài 3-grams và tính df/IDF của hai feature. Sau đó đọc `outputs/stage2/baseline/comparison.csv` để giải thích vì sao C được chọn. Không fit lại baseline chỉ để xem kết quả đã có.

9. PhoBERT — Mô hình học biểu diễn ngữ cảnh

PhoBERT là encoder ngôn ngữ đã tiền huấn luyện; project tải `vinai/phobert-base` rồi fine-tune cho phân loại. Tiền huấn luyện cung cấp biểu diễn ngôn ngữ trước khi học nhãn ViHSD. Fine-tuning điều chỉnh encoder và classification head theo loss của bài toán; cấu hình mặc định `freeze_epochs=0` không đóng băng encoder ở đầu training.

9.1. Transformer ở mức đủ để đọc project

Token ID được ánh xạ thành embedding. Thông tin vị trí giúp phân biệt thứ tự. Trong self-attention, biểu diễn mỗi vị trí tạo Query, Key, Value:

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

Điểm Query–Key quyết định cách trộn Value giữa các vị trí. Nhiều heads học các quan hệ khác nhau; feed-forward, residual connection và normalization hỗ trợ biến đổi và tối ưu. Không cần tự cài lại Transformer để hiểu wrapper hiện tại: phần encoder do thư viện thực hiện.

9.2. Head thực tế trong `src/model.py`

```
input_ids, attention_mask                [B, L]
encoder.last_hidden_state                [B, L, H]
lấy vị trí đầu: [:, 0]                   [B, H]
Dropout → Linear(H,H) → Tanh → Dropout
Linear(H,3)                              [B, 3] logits
```

`H` đọc từ `encoder.config.hidden_size`; không phải code tự hardcode kích thước. Project dùng hidden state vị trí đầu, không mean-pooling mọi token và không dùng pooler layer. `forward` trả logits, chưa softmax.

Với logits `z`, softmax chuyển thành ba số tổng bằng 1:

```
p_k = exp(z_k) / sum_j exp(z_j)
prediction = argmax_k p_k
confidence = max_k p_k
```

Logits `[2, 1, 0]` cho xác suất xấp xỉ `[0.665, 0.245, 0.090]`, nên nhãn là CLEAN. Đây là ví dụ toán học, không phải dự đoán thực của checkpoint trên một câu cụ thể.

Confidence 0.9 không tự động nghĩa "90% khả năng quyết định đúng" trên dữ liệu thực. Muốn diễn giải như tần suất đúng cần kiểm tra calibration. Model hiện chưa được hiệu chuẩn.

Thực hành: tự viết softmax bằng NumPy, trừ `max(z)` trước exp để giảm overflow; so với phép tính tay. Giải thích vì sao `argmax(logits)` và `argmax(softmax(logits))` chọn cùng lớp khi giá trị hữu hạn.

10. Training — Model thay đổi thế nào?

10.1. Loss và mất cân bằng lớp

Cross-entropy của một mẫu nhãn y là `-log(p_y)`: dự đoán tự tin nhưng sai bị phạt mạnh. PyTorch CrossEntropy nhận logits và tự xử lý phần log-softmax, nên không softmax trước khi truyền vào loss.

Project hỗ trợ ba lựa chọn `imbalance`: `none`, `weighted_loss`, `sampler`. Class weights lấy từ train thực dùng:

```
w_k = N / (K × n_k), K = 3
weighted mean loss = sum_i w_(y_i) × [-log p_i(y_i)] / sum_i w_(y_i)
```

Lớp ít mẫu có trọng số cao hơn. Weighted loss tăng mức đóng góp lỗi của mẫu; sampler thay đổi tần suất mẫu được rút, có hoàn lại. Code chọn một chiến lược, không đồng thời bật cả hai. Không cân bằng lại validation/test để tạo điểm đẹp hơn.

10.2. Một vòng cập nhật trong `train_epoch`

1. `model.train()` bật hành vi training, gồm dropout; đưa batch tới device.
2. Forward lấy logits và tính weighted loss dạng tổng.
3. `backward()` dùng chain rule tính gradient, cộng dồn vào gradient đang có.
4. Khi đủ nhóm accumulation hoặc tới batch cuối: unscale gradient nếu dùng FP16.
5. Chia gradient cho tổng trọng số thực trong nhóm; clipping theo norm.
6. Optimizer cập nhật tham số, scaler cập nhật hệ số, scheduler tiến khi optimizer step không bị bỏ vì overflow.
7. `zero_grad` chuẩn bị nhóm tiếp theo.

Epoch là một lượt qua DataLoader; **microbatch** là nhóm xử lý một forward/backward; **optimizer step** là lần cập nhật trọng số. Với sampler có hoàn lại, một epoch không đảm bảo mỗi hàng xuất hiện đúng một lần.

Effective batch xấp xỉ `microbatch × accumulation`: model được chọn dùng `8 × 2 = 16`. Nhóm cuối có thể ít hơn. Chia loss mỗi microbatch cho cùng một hằng số có thể sai khi batch cuối ngắn hoặc class weights khác; code chuẩn hóa theo tổng trọng số thực. Test tương đương large batch kiểm tra toán gradient trong điều kiện kiểm soát, không bảo đảm hai run thực có dropout luôn trùng bit.

10.3. Những công cụ hỗ trợ training

Cơ chế	Vai trò	Điều cần hiểu
AdamW	Dùng thông tin gradient tích lũy và weight decay để cập nhật	Learning rate quá lớn/nhỏ đều có thể bất lợi
Weight decay	Hạn chế trọng số lớn	Code tách bias/parameter 1 chiều khỏi decay

Cơ chế	Vai trò	Điều cần hiểu
Warmup + cosine	Tăng LR ban đầu rồi giảm theo lịch	Scheduler đếm optimizer steps
Dropout	Bỏ ngẫu nhiên một phần activation khi train	Tắt khi `eval()`
Gradient clipping	Giới hạn norm gradient	Không sửa được mọi nguyên nhân loss NaN
FP16 + GradScaler	Giảm chi phí trên GPU phù hợp	Scaler hỗ trợ tránh gradient quá nhỏ; CPU yêu cầu FP32
Gradient checkpointing	Tính lại activation lúc backward để giảm RAM/VRAM	Đổi thêm tính toán lấy bộ nhớ
Early stopping	Dừng khi metric không cải thiện đủ qua patience	Project mặc định chọn Macro-F1, không phải train loss
Freeze/unfreeze	Có thể khóa encoder vài epoch rồi mở	Có hỗ trợ trong code, không có nghĩa run cuối đã dùng

`evaluate` dùng `model.eval()` và không tính gradient. Validation vẫn đo loss với weights được truyền từ train, đồng thời tính Macro-F1 từ dự đoán; Macro-F1 không bị nhân class weights của loss.

Thực hành: đọc `test_weighted_accumulation_matches_large_batch`, `test_early_stopping`. Với patience=2, hãy tự mô phỏng chuỗi Macro-F1 `0.60, 0.62, 0.619, 0.618` và nêu best epoch, thời điểm dừng khi min_delta=0.0001.

11. Thí nghiệm, tuning và tái lập

11.1. Config mẫu không phải cấu hình model cuối

Tham số	`configs/phobert_colab.json`	Model được chọn trong báo cáo
Learning rate	2e-5	3e-5
Max length	128	128
Dropout	0.1	0.1
Microbatch	8	8
Accumulation	4	2
Effective batch	32	16
Seed	42	42

Run cuối là `phobert_lr3e-5_len128_drop01_batch16_seed42`, best epoch 5. Không chạy config mẫu rồi nghĩ rằng đã tái lập đúng run này.

Tuning đã thực hiện tám cấu hình tuần tự: bốn learning rates, một length 256, hai dropout khác và một effective batch 16. Đây không phải full grid của mọi tổ hợp. `tune.py` có khả năng sinh nhiều candidates hơn theo budget; khả năng code không đồng nghĩa tất cả đã được chạy. Các bước tuning thủ công trong notebook và bảng cuối cũng cần đọc cùng kết quả search.

Giữ seed 42 hỗ trợ so sánh trong giao thức hiện tại, nhưng không đo biến thiên qua nhiều seeds. Tuning tuần tự có thể bỏ lỡ tương tác, chẳng hạn dropout tốt nhất ở một LR không chắc tốt nhất ở LR khác.

11.2. Artifact nào dùng làm gì?

Artifact	Nội dung và mục đích
`config.json`	Các lựa chọn của run
`manifest.json`	Hash data/source/assets, số mẫu, môi trường và revision
`history.json`	Metrics, LR, thời gian theo epoch
`best_model.pt`	Weights tại checkpoint được chọn theo validation
`last_checkpoint.pt`	Weights và trạng thái optimizer/scheduler/scaler/RNG để resume
`tokenizer/`, `encoder_config/`	Dựng lại kiến trúc và tokenizer offline
`validation_predictions.csv`	Dự đoán có row ID để tính lại/đối chiếu
`source_snapshot/`	Source lịch sử để truy nguyên run

Reload nạp model để inference. **Resume** tiếp tục training từ cuối epoch đã hoàn tất; cần nhiều hơn weights. **Reproduce** chạy lại một quy trình và kiểm tra mức độ tương đồng, phụ thuộc data/code/assets/môi trường/randomness.

Một chi tiết đọc checkpoint: best lưu `epoch` theo số thứ tự bắt đầu từ 1; last trong training lưu chỉ số vòng lặp bắt đầu từ 0. Phải đọc cách producer/consumer chuyển đổi trước khi kết luận checkpoint lệch một epoch.

Guard resume kiểm tra cấu hình liên quan, hashes dữ liệu/source và assets. Thay source/hyperparameters thì tạo run mới, không bỏ guard. Seed đã đặt cho Python, NumPy, Torch và các generator liên quan; vẫn không nên hứa bitwise reproducibility trên mọi phần cứng/thư viện.

File tạm + `os.replace` giúp tránh để file đích ở trạng thái ghi dở; không biến toàn bộ thư mục run thành một transaction và không đảm bảo OneDrive/Drive đã đồng bộ xong tất cả file.

Bài học thực tế CKPT-02: báo cáo hiện ghi last checkpoint rerun length 256 mới tới epoch 4 trong khi best/history tới epoch 5. Best đã reload và khớp toàn bộ validation; điều đó không chứng minh last checkpoint cũng đầy đủ. Cần đối chiếu Drive/hash; chưa biết nguyên nhân.

12. Evaluation — Đo chất lượng đúng cách

12.1. Confusion matrix và metrics

Project dùng hàng là nhãn thật, cột là dự đoán; thứ tự CLEAN, OFFENSIVE, HATE. Với một lớp k, xem k là positive và hai lớp còn lại là negative:

```
precision_k = TP_k / (TP_k + FP_k)
recall_k    = TP_k / (TP_k + FN_k)
F1_k        = 2 TP_k / (2 TP_k + FP_k + FN_k)
Macro-F1    = (F1_CLEAN + F1_OFFENSIVE + F1_HATE) / 3
Weighted-F1 = sum_k (support_k / N) × F1_k
Accuracy    = số dự đoán đúng / N
```

Precision hỏi “trong những mẫu gọi là k, bao nhiêu đúng?”; recall hỏi “trong những mẫu thật là k, tìm được bao nhiêu?”. Project dùng `zero_division=0` và danh sách đủ ba lớp, kể cả khi lát cắt thiếu một lớp.

Ví dụ tổng hợp:

```
               dự đoán
              C   O   H
thật CLEAN    8   1   1
```

```
thật OFFENSIVE  2   2   0
thật HATE       1   1   4
```

Accuracy = 14/20 = 0.70. F1 lần lượt khoảng 0.7619, 0.5000, 0.7273; Macro-F1 khoảng 0.6631. Với OFFENSIVE, TP=2, FP=2, FN=2. Tự tính Weighted-F1 khoảng 0.6991 và giải thích vì sao không bằng Macro-F1.

Vì CLEAN chiếm đa số, accuracy/Weighted-F1 có thể cao trong khi OFFENSIVE kém. Macro-F1 cho mỗi lớp một trọng số ngang nhau nên được dùng chọn model.

12.2. Kết quả đã lưu của project

Split	Model	Macro-F1	Weighted-F1	Accuracy
Validation	Baseline Colab	0.6173	0.8380	0.8335
Validation	PhoBERT được chọn	0.6843	0.8691	0.8679
Test	Baseline Colab	0.6325	0.8494	0.8436
Test	PhoBERT được chọn	0.6546	0.8665	0.8647

Không gọi 0.6843 là test Macro-F1. Test PhoBERT tăng khoảng 0.0221 điểm tuyệt đối so với baseline, tương đương 2.21 điểm phần trăm trên thang phần trăm; không phải tăng 2.21% tương đối.

Test có 5.548 CLEAN, 444 OFFENSIVE, 688 HATE. PhoBERT có OFFENSIVE F1 0.4162 và HATE recall 0.5770. Điểm tổng hợp tốt hơn baseline không có nghĩa đã đạt mọi yêu cầu cho từng lớp.

12.3. Bootstrap đo độ bất định nào?

`paired_bootstrap` lấy mẫu lại chỉ số dòng có hoàn lại 2.000 lần bằng NumPy. Mỗi lần dùng cùng tập chỉ số cho hai model, tính Macro-F1 mỗi model và chênh lệch; lấy quantile 2.5%/97.5%.

“Paired” giữ quan hệ rằng hai model dự đoán trên cùng một ví dụ. Lấy hai mẫu chỉ số độc lập sẽ mất cấu trúc so sánh này. Kernel dùng `np.bincount(3*y + p).reshape(3,3)` tạo confusion matrix, rồi tính F1 bằng phép toán array.

Báo cáo hiện có khoảng 95% cho Macro-F1 PhoBERT xấp xỉ `[0.6360, 0.6728]`; chênh lệch PhoBERT–baseline `[0.0026, 0.0418]`. Khoảng này điều kiện trên các model cố định và quy trình resampling theo dòng. Nó không đo bất định do training seed; duplicate hoặc ngữ cảnh chung có thể vi phạm giả định độc lập và làm khoảng quá hẹp.

Thực hành: đọc `test_paired_bootstrap_preserves_pairing_and_reproducibility`; giải thích vì sao khi hai mảng dự đoán giống nhau, chênh lệch ở mỗi bootstrap sample phải bằng 0.

13. Phân tích lỗi — Biến con số thành hiểu biết

PhoBERT sai 904/6.680 mẫu test. Báo cáo chỉ ra các hướng nhầm, ví dụ CLEAN → OFFENSIVE 234 mẫu và HATE → CLEAN 189 mẫu. Confusion matrix cho biết **nhầm gì**, chưa đủ biết **vì sao nhầm**.

Mở `outputs/stage3/qualitative_analysis.md`, `errors.csv` và `error_review.csv`. Review queue chọn tối đa 10 lỗi confidence cao mỗi hướng nhầm, nên không phải mẫu ngẫu nhiên đại diện toàn bộ lỗi. Với mỗi ví dụ, ghi riêng:

1. Sự kiện quan sát được: nhãn thật, nhãn dự đoán, confidence, có truncation hay không.

2. Giả thuyết: mỉa mai, trích dẫn, từ mơ hồ, thiếu ngữ cảnh, lỗi annotation...

3. Bằng chứng cần bổ sung: ngữ cảnh hoặc hướng dẫn gán nhãn.

Không chuyển một giả thuyết thành kết luận “nhãn sai” nếu chưa review đủ. Không sửa nhãn test hoặc chọn threshold theo các lỗi đã xem.

Validation/test overlap hiện tương ứng 95 dòng test. Lát cắt không overlap có Macro-F1 khoảng 0.6536; đây là phân tích độ nhạy, không thay điểm test chính. Chỉ ba mẫu test dài hơn 128 tokens: lát cắt này quá nhỏ để khái quát rằng tăng length sẽ tốt hơn hoặc tệ hơn.

Vì sao validation loss tăng mà F1 vẫn tăng?

Loss quan tâm giá trị xác suất, còn F1 quan tâm nhãn argmax. Model có thể sửa đúng thêm một số nhãn nhưng trở nên rất tự tin ở vài lỗi còn lại; cross-entropy tăng mạnh trong khi F1 tăng. Trong run cuối, train loss giảm và validation loss tăng ở các epoch sau là dấu hiệu overfitting; best epoch 5 vẫn đúng tiêu chí validation Macro-F1 đã chọn trước.

Thực hành: chọn một ví dụ tổng hợp, giữ nguyên argmax sai nhưng tăng confidence của lớp sai, tính lại cross-entropy. Giải thích tại sao accuracy không đổi mà loss tăng. Viết một đoạn nhận xét kết quả project có đủ split, metric, seed và giới hạn.

14. Stage 4 — Từ model thành API

14.1. Bundle inference

`scripts/package_stage4.py` đóng gói model đã khóa cùng tokenizer, encoder config, preprocessing và VnCoreNLP. `outputs/stage4/bundle/` không cần train/test để phục vụ. Bundle không được lưu cùng Git như source; cần chuyển riêng khi triển khai.

`verify_bundle` kiểm tra schema, nhãn, danh sách file cần thiết, hash và đường dẫn phải nằm trong bundle. Việc này phát hiện thiếu/sai artifact và đường dẫn thoát thư mục. Manifest hash hữu ích khi manifest đáng tin cậy; nó không phải chữ ký xác thực từ một bên độc lập.

Trên Windows có path tiếng Việt, serving sao chép assets VnCoreNLP vào TEMP ASCII và kiểm tra lại hash để tránh vấn đề classpath đã gặp. Đây là xử lý môi trường, không thay thuật toán phân loại.

14.2. Vòng đời và đồng thời

FastAPI định nghĩa hợp đồng HTTP; Uvicorn chạy ứng dụng ASGI. Lifespan tạo một worker thread, nạp model/segmenter, warmup rồi đánh ready. Nạp một lần tránh chi phí khởi tạo ở mọi request.

Inference và JVM chạy trên cùng worker chuyên dụng. Mỗi process nhận một request inference tại một thời điểm; request mới lúc bận nhận 503 và `Retry-After: 1`. Event loop vẫn có thể xử lý health route khi inference chạy ở thread khác. Client hủy request không làm slot được giải phóng trong khi model còn chạy, nhờ shield/future callback.

Chạy Uvicorn `--workers 1` để không tạo nhiều bản model/JVM ngoài dự kiến. “Async API” không đồng nghĩa nhiều forward GPU/CPU được chạy đồng thời, và số CPU threads của Torch là khái niệm khác số Uvicorn workers.

14.3. Hợp đồng HTTP

Route	Mục đích
`GET /health`	Kiểm tra ứng dụng phản hồi khi route được phục vụ

Route	Mục đích
`GET /ready`	Trạng thái sẵn sàng và bận của model
`GET /model`	Run, hash checkpoint, device, giới hạn, calibration
`POST /predict`	Nhận `{"text":"Xin chào!"}`
`POST /predict/batch`	Nhận `{"texts":["Xin chào!","Cảm ơn bạn"]}`
`GET /docs`, `/openapi.json`	Giao diện thử và schema API

Pydantic kiểm tra kiểu/độ dài, không chấp nhận field thừa. Mỗi câu 1–5.000 ký tự, mỗi batch 1–32 câu và tổng tối đa 20.000 ký tự; body tối đa 128 KiB kiểm tra trước decode JSON. Byte khác ký tự, nhất là Unicode tiếng Việt. Text chỉ chứa whitespace hoặc ký tự surrogate lỗi bị từ chối.

Kết quả gồm `label_id`, `label`, `confidence`, `probabilities`, `token_count`, `truncated`. Token count đo trước truncation và gồm special tokens; batch giữ thứ tự. API không trả lại nội dung câu trong prediction.

Status	Diễn giải
200	Thành công
422	Sai schema/giá trị đầu vào
413	Vượt giới hạn body
503	Chưa ready hoặc đang bận
500	Inference lỗi, không trả traceback cho client

14.4. Chạy thử có mục đích

Sau khi đã có môi trường Stage 4, Java và bundle hợp lệ:

```
python -m uvicorn src.api:app --host 127.0.0.1 --port 8000 --workers 1 --no-access-log
```

Ở terminal khác:

```
$payload = @{text = 'Chúc bạn một ngày vui vẻ!'} | ConvertTo-Json
Invoke-RestMethod -Uri http://127.0.0.1:8000/predict -Method Post -ContentType 'application/json; chars
    et=utf-8' -Body ([System.Text.Encoding]::UTF8.GetBytes($payload))
```

Đừng yêu cầu ví dụ này bắt buộc trả CLEAN như một định luật; ghi lại dự đoán thật và kiểm tra hợp đồng: nhãn thuộc ba lớp, probabilities tổng gần 1, confidence là giá trị lớn nhất, single/batch nhất quán.

Dockerfile/Compose đã chuẩn bị nhưng chưa được xác minh build/run trong trạng thái báo cáo hiện tại. Image là môi trường đóng gói, container là instance chạy; Compose cấu hình service và mount bundle read-only. Cấu hình sẵn không đồng nghĩa đã deploy. Hướng dẫn đầy đủ: `docs/STAGE4_API.md`.

15. Tests, verifiers và bằng chứng

Tầng	Kiểm tra điều gì?	Không chứng minh điều gì?
Unit/regression tests	Hành vi hàm và tình huống lỗi với fixture	Chất lượng ngôn ngữ trên thực tế
Artifact verifier	Hash, row alignment, metrics từ dự đoán đã lưu	Mọi checkpoint đều inference đúng nếu không reload

Tầng	Kiểm tra điều gì?	Không chứng minh điều gì?
Reload inference	Checkpoint/tokenizer/pipeline tạo lại dự đoán	Resume optimizer state đầy đủ
HTTP verifier model thật	Request đi qua server và inference thật	Khả năng chịu tải public trên máy khác
Test-set evaluation	Chất lượng trên tập test và giao thức đã ghi	Hiệu quả trên mọi miền/ngữ cảnh mới

Các tests nên đọc theo thứ tự:

• Stage 1: Unicode/idempotence, missing values, class weights, exclusions và holdout integrity.

• Stage 2: padding/truncation, split guard, early stopping, cache invalidation, accumulation, resume.

• Stage 3: paired bootstrap, overlap alignment, duplicate IDs, evaluation lock.

• Stage 4: load once, schema, body limit, busy/health, cancellation, probabilities và bundle path escape.

API tests có dùng predictor giả để chạy nhanh và cô lập hành vi HTTP. `scripts/verify_stage4.py` dùng model/VnCoreNLP thật, đối chiếu 12 mẫu validation; 12/12 khớp không phải accuracy trên toàn bộ dữ liệu.

Chọn lệnh phù hợp với mục đích

```
# Tests với môi trường có đủ dependency kiểm thử
python -m pytest -q

# Đối chiếu artifact Stage 3, không inference lại
python scripts/verify_stage3.py

# HTTP verification: tự chạy server và ghi báo cáo kiểm chứng
python scripts/verify_stage4.py
```

`verify_stage2.py` chỉ reload 12 mẫu của run ban đầu; `audit_downloaded_stage2.py` kiểm tra artifact Colab và reload toàn bộ validation model cuối; `verify_len256_rerun.py` kiểm tra riêng rerun và có thể trả exit 1 vì last checkpoint không đầy đủ dù best inference đạt. Đọc phạm vi script trước khi diễn giải PASS/FAIL.

Các lệnh tái tạo như `python -m src.stage1`, đóng gói `python scripts/package_stage4.py` và các verifier ghi báo cáo có thể thay output. Không cần chạy lại toàn bộ khi mục tiêu chỉ là đọc/học kết quả lịch sử. Training mới dùng config/output riêng; hướng dẫn Colab tại `docs/STAGE2_COLAB.md`.

Nếu sửa code, cập nhật `AI_TRACKER.md` với thay đổi, lỗi chưa giải quyết và kiểm chứng thật. Không đánh dấu “đã test” khi chỉ đọc test. Khi báo lỗi, giữ traceback, lệnh chạy, phiên bản và input tái hiện tối thiểu.

16. Bộ bài tập “vừa làm vừa học”

Bài	Việc tự làm	Tiêu chí kiểm tra
A — Đọc pipeline	Vẽ hai luồng train và inference, ghi tên hàm ở mỗi bước	Phân biệt được chỗ fit, chỗ predict và nơi dùng nhãn
B — Preprocessing	Lập 8 câu tổng hợp có dấu/emoji/URL/mention/whitespace	Dự đoán và đối chiếu output; không mất emoji ngoài dự kiến
C — Data audit	Tạo ví dụ trên giấy có duplicate, conflict và overlap	Giải thích chính xác hàng bị loại và lý do chính
D — Class weights	Với counts `[80, 10, 10]`, tính weights	Kết quả `[0.4167, 3.3333, 3.3333]` xấp xỉ
E — Tensor	Vẽ shape qua encoder và classification head	Đầu ra cuối `[B,3]`, không `[B,L,3]`

Bài	Việc tự làm	Tiêu chí kiểm tra
F — Loss	Tính `-log(p_y)` cho 0.8, 0.2, 0.01	Xác suất nhãn đúng càng thấp, loss càng lớn
G — Metrics	Tính tay confusion matrix ví dụ phần 12	Macro-F1 gần 0.6631; giải thích FP/FN đúng
H — Checkpoint	So metadata best/last/history của một run bằng công cụ đáng tin	Giải thích index epoch, reload và resume
I — Error review	Viết 5 nhận xét từ review queue	Tách quan sát khỏi giả thuyết; không sửa test
J — API	Thử single/batch, input rỗng, sai kiểu và câu dài	Ghi status/body; giải thích truncation khác input rejection
K — Tái lập	Lập danh sách file để mang inference sang máy khác	Có weights, config, tokenizer, preprocessing, segmenter và môi trường
L — Thuyết trình	Trình bày project trong 10 phút không đọc source	Nêu được baseline, kết quả đúng split, giới hạn và bằng chứng

Mẫu ghi chép cho mỗi bài:

```
Câu hỏi cần hiểu:
Hàm/file đã đọc:
Dự đoán trước khi chạy:
Lệnh hoặc thao tác thực hiện:
Kết quả quan sát được:
Giải thích vì sao:
Điều còn chưa hiểu:
Bằng chứng: đường dẫn output, test hoặc trích đoạn nhỏ
```

Nếu muốn thực hiện một nghiên cứu mới, viết giả thuyết và cách đánh giá trước. Với test hiện tại đã xem, không dùng nó để tối ưu tiếp rồi coi là test độc lập. Cần giao thức đánh giá mới phù hợp; không tự thay đổi kết quả đã khóa. Nhiều seeds là một hướng nghiên cứu tùy chọn, không phải yêu cầu bổ sung cho lần hoàn tất hiện tại.

17. Những chỗ dễ mắc lỗi và cách tìm nguyên nhân

Hiện tượng	Kiểm tra trước	Bài học
Không import được package	Python executable, môi trường, requirements đúng stage	Cài thư viện vào Python khác không giúp process đang chạy
VnCoreNLP không khởi tạo	Java, assets, path/classpath, thông báo lỗi	Python package không chứa toàn bộ Java runtime
CUDA unavailable	Runtime Colab có GPU, bản Torch, config device	GPU không được bật chỉ bởi tên config
CPU báo precision	Chọn FP32 cho CPU	Cấu hình FP16 ở đây dành cho GPU phù hợp
CUDA out of memory	Microbatch, length, checkpointing, process còn chiếm VRAM	Tăng accumulation không giảm bộ nhớ của một microbatch
Search plan changed	Config và thư mục search lịch sử	Dùng output mới thay vì bỏ guard
Source changed khi resume	Hash source so với run	Source mới cần run mới
Validation tốt nhưng test giảm	Generalization, lựa chọn nhiều lần trên validation, split	Không kết luận lỗi code chỉ từ score giảm

Hiện tượng	Kiểm tra trước	Bài học
API confidence cao nhưng sai	Calibration, miền dữ liệu, ngữ cảnh	Confidence không thay thế đánh giá
503 khi gọi API dồn dập	Trạng thái busy/ready	Thiết kế hiện chủ động từ chối khi bận

Những giới hạn đang được theo dõi: last checkpoint rerun length 256 chưa đủ; môi trường deserialize baseline khác Colab; mapping Drive/local; archive cũ; Docker/public chưa xác minh. Xem `AI_TRACKER.md` để biết trạng thái cập nhật, không tự xem chúng là đã giải quyết chỉ vì có hướng dẫn trong tài liệu này.

18. Tự đánh giá và thứ tự đọc tiếp

Chỉ bạn đánh dấu khi có thể tự giải thích hoặc thực hành được:

[ ] Tôi mô tả được bài toán ba lớp và giới hạn của nhãn.

[ ] Tôi phân biệt train, validation, test và các hình thức leakage.

[ ] Tôi giải thích được vì sao train còn 21.292 mẫu.

[ ] Tôi hiểu preprocessing, fingerprint, segmentation và tokenization khác nhau.

[ ] Tôi theo dõi được một câu đến tensor và logits.

[ ] Tôi giải thích được TF-IDF, Logistic Regression và vai trò baseline.

[ ] Tôi hiểu head PhoBERT và softmax không phải calibration.

[ ] Tôi giải thích được weighted loss, gradient accumulation và optimizer step.

[ ] Tôi đọc được config/history và phân biệt best với last checkpoint.

[ ] Tôi tính được precision, recall, F1 và Macro-F1 từ confusion matrix.

[ ] Tôi giải thích được paired bootstrap và giới hạn theo dòng/một seed.

[ ] Tôi viết được phân tích lỗi mà không suy diễn nguyên nhân thiếu bằng chứng.

[ ] Tôi giải thích được request API đi qua những bước nào.

[ ] Tôi phân biệt unit tests, reload verification, benchmark và chất lượng model.

[ ] Tôi trình bày đúng 0.6843 là validation, 0.6546 là test Macro-F1.

[ ] Tôi biết thay đổi nào cần run mới và cách ghi bằng chứng vào tracker.

Đọc tiếp trong project theo nhu cầu:

Tài liệu/artifact	Khi nào đọc
`README.md`	Tóm tắt trạng thái và lệnh chính
`docs/PROJECT_STRUCTURE.md`	Tìm vị trí file, migration và đường dẫn
`docs/STAGE2_COLAB.md`	Thực hiện training/tuning trên Colab
`outputs/stage2/review/summary.md`	Hiểu checkpoint nào đã được kiểm chứng
`docs/STAGE3_EVALUATION.md`	Giao thức test và cách chạy evaluation

Tài liệu/artifact	Khi nào đọc
`outputs/stage3/summary.md`	Metrics, confusion matrix, bootstrap và learning curves
`outputs/stage3/qualitative_analysis.md`	Phân tích ngữ cảnh của lỗi
`docs/STAGE4_API.md`	Cài/chạy API, bundle và Docker
`AI_TRACKER.md`	Lịch sử, lỗi còn lại và đề xuất chưa thực hiện

Phạm vi tài liệu bao phủ pipeline hiện có và kiến thức để đọc, vận hành, kiểm chứng, mở rộng nó. Các chủ đề học sâu tiếp theo là tokenizer BPE, tối ưu AdamW, calibration, domain shift và thiết kế annotation; chúng chưa phải tính năng được triển khai thêm chỉ bởi việc nhắc tới ở đây.